

EC-341 Digital System Design

BS (CE)-2021

LAB REPORT # 11



Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz
21-CE-020	Muhammad Ibthesam
21-CE-021	Yawar Ali Malik

Department of Computer Engineering

HITEC University Taxila

	Presentation (Format(0.5)+ Title (0.5)+ Conclusion(1) +Procedure(2) +Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	5	5	5	15
Obtained				

Lab Instructor.

Fasih Ahmad

Experiment #11

Implementation of Mealy State Machine

Objective:

1. State Machine as a Controller
2. Design of Mealy State Machine
3. Overlapping and Non-Overlapping
4. Example
5. Lab Tasks

Equipment/Software Used:

Xilinx Vivado

Procedure:

A state machine is an abstract way of describing the behavior of a controller. The state machine description reduces the description of any controller to just four things: inputs, outputs, states, and state transition rules. The state machine description contains no details about the implementation of the controller (no flowcharts, programs, schematics, mechanical diagrams, etc.). Instead, a state machine only describes the different states that the controller can be in, the outputs in each state, and what input conditions will cause transitions between states. A state machine description can be used to describe and analyze existing systems and to aid in the design of new systems. The simplified description represented by a state machine allows us to verify proper operation and document the logical operation of a controller before proceeding with detailed design (e.g. writing a program or designing hardware).

Design of State Machine:

There are four steps to designing a state machine:

1. define the outputs – these are the output signals used by the controller
2. define the inputs – the input signals provided to the controller
3. define all of the possible states that the controller will need to be in and what values the outputs will take for each of these states
4. define the exact input conditions that will cause the state machine to switch from one state to another.

Inputs and Outputs:

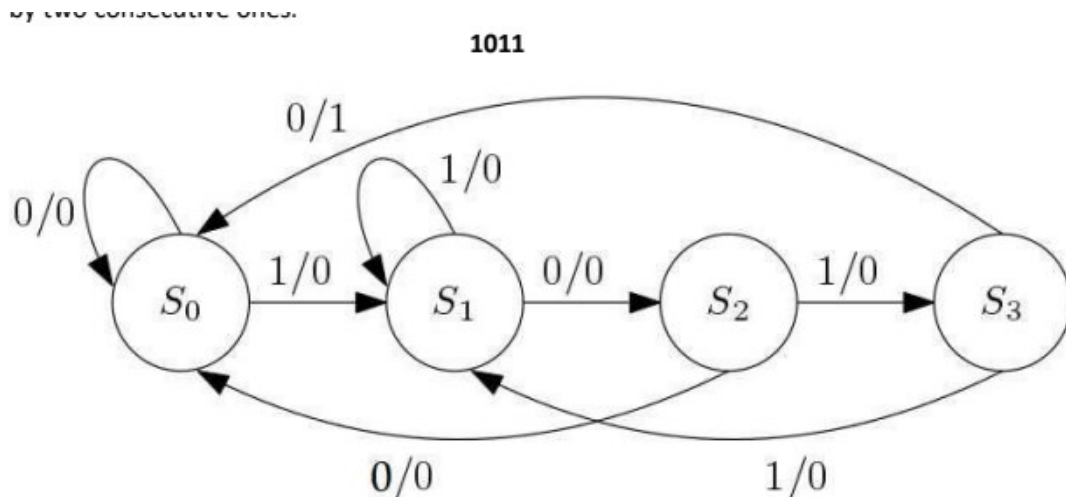
A controller's inputs are typically signals from sensors such as switches, thermometers, pressure gauges, and so on. The outputs are signals that drive actuators such as motors, valves, displays, etc. We will restrict ourselves to inputs and outputs that are binary values (represented as 0/1, false/true, or low/high). To simplify the state machine description, each input and output is given an abbreviation, usually mnemonic. For example an input that indicates that a tank is full might be given the label full, an output that turns a drill motor on might be given the label drill.

Tasks

Task #1:

Execute the code, given in example for overlapping/non-overlapping case.

STATE DIAGRAM:



Truth Table:

Present State	Input	Next State	Output
S0	1	S1	0
S0	0	S0	0
S1	1	S1	0
S1	0	S2	0
S2	1	S3	0
S2	0	S0	0
S3	1	S1	1
S3	0	S2	0

Verliog:

```
module mealy(  
    input din,  
    input clk,  
    input reset,  
    output reg y  
);  
    reg [1:0] cst, nst;  
    parameter S0 = 2'b00,  
    S1 = 2'b01,  
    S2 = 2'b10,  
    S3 = 2'b11;  
    always@(posedge clk)  
    begin  
        if (reset)  
            cst <= S0;  
        else  
            cst <= nst;  
        end  
    always @(cst or din)  
    begin  
        case (cst)  
            S0: if (din == 1'b1)  
                begin  
                    nst = S1;
```

```
y=1'b0;
end
else
begin
nst = cst;
y=1'b0;
end
S1: if (din == 1'b0)
begin
nst = S2;
y=1'b0;
end
else
begin
y=1'b0;
nst = cst;
end
S2: if (din == 1'b1)
begin
nst = S3;
y=1'b0;
end
else
begin
```

```

    nst = S0;
    y=1'b0;
end
S3: if (din == 1'b0)
begin
    nst = S0;
    y=1'b1;
end
else
begin
    nst = S1;
    y=1'b0;
end
default: nst = S0;
endcase
end

endmodule
module testbench();
reg sequence_in;
reg clock;
reg reset;
wire detector_out;
mealy uut (.din(sequence_in),.clk(clock),.reset(reset),.y(detector_out));
initial
begin

```

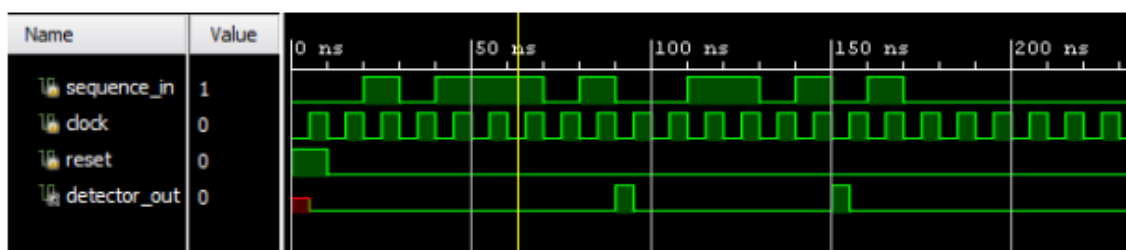
```
clock = 0;
forever #5 clock = ~clock;
end
initial
begin
sequence_in = 0;
reset = 1;
#10;
reset = 0;
#10;
sequence_in = 1;
#10;
sequence_in = 0;
#10;
sequence_in = 1;
#30;
sequence_in = 0;
#10;
sequence_in = 1;
#10;
sequence_in = 0;
#10;
sequence_in = 0;
#10;
```

```

sequence_in = 1;
#10;
sequence_in = 1;
#10
sequence_in = 0;
#10;
sequence_in = 1;
#10;
sequence_in = 0;
#10;
sequence_in = 1;
#10
sequence_in = 0;
end
endmodule

```

Waveform:



Task #2:

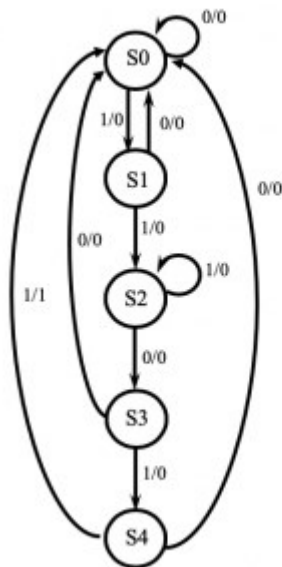
You are required to write code for a basic sequential state machine using mealy based design, which detects 11011 for the following cases:

a. Overlapping

b. Non-Overlapping

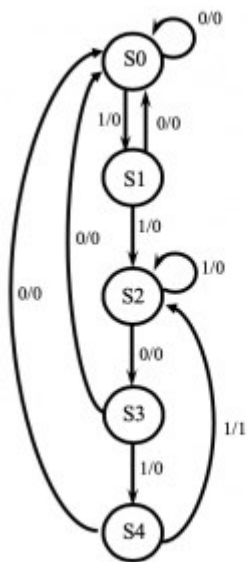
STATE DIAGRAM:

Mealy machine (Non-Overlapping) of 11011



STATE DIAGRAM:

Mealy machine (Overlapping) of 11011



VERLIQG CODE:

```
module sequence_detector(
```

```
    input clk,
```

```
    input reset,
```

```

input data,
output reg detected
);
parameter IDLE = 2'b00;
parameter STATE1 = 2'b01;
parameter STATE2 = 2'b10;
parameter STATE3 = 2'b11;
reg [1:0] state, next_state;
always @(posedge clk or posedge reset) begin
    if (reset)
        begin
            state <= IDLE;
        end
    else
        begin
            state <= next_state;
        end
end
always @(*) begin
    case(state)
        IDLE: begin
            if (data)
                begin
                    next_state = STATE1;

```

```
    end
    else
    begin
        next_state = IDLE;
    end
    detected = 0;
end
STATE1: begin
    if (data) begin
        next_state = STATE2;
    end else begin
        next_state = IDLE;
    end
    detected = 0;
end
STATE2: begin
    if (data) begin
        next_state = STATE3;
    end else begin
        next_state = IDLE;
    end
    detected = 0;
end
STATE3: begin
```

```

        if (data) begin
            next_state = STATE2;
            detected = 0;
        end else begin
            next_state = STATE1;
            detected = 0;
        end
        if (state == STATE2 && data) begin
            detected = 1;
        end
    end
endcase
end

endmodule

module sequence_detector_tb();
parameter CLK_PERIOD = 10; // Clock period in ns

reg clk;
reg reset;
reg data;
wire detected;

sequence_detector uut (
    .clk(clk),
    .reset(reset),
    .data(data),

```

```
.detected(detected)

);

always #((CLK_PERIOD/2)) clk = ~clk;

initial

begin

    clk = 0;

    reset = 1;

    data = 0;

    #20; // Wait for 20 ns

    reset = 0; // Deassert reset


    // Test case 1: Non-overlapping pattern

    #10; data = 1;

    #10; data = 1;

    #10; data = 0;

    #10; data = 1;

    #10; data = 1;

    #10; data = 0;

    #10; data = 0;

    #10; data = 1;

    #10; data = 1;

    #10; data = 1;

    #10; data = 0;

    #10; data = 1;
```

```
#10; data = 0;
```

```
#10; data = 1;
```

```
#10; data = 1;
```

```
#10; data = 1;
```

```
#10; data = 0;
```

```
#10; data = 1;
```

```
#10; data = 1;
```

```
#10; data = 1;
```

```
#10; data = 0;
```

```
#10; data = 1;
```

```
#10; data = 1;
```

```
#10; data = 0;
```

```
#10; data = 1;
```

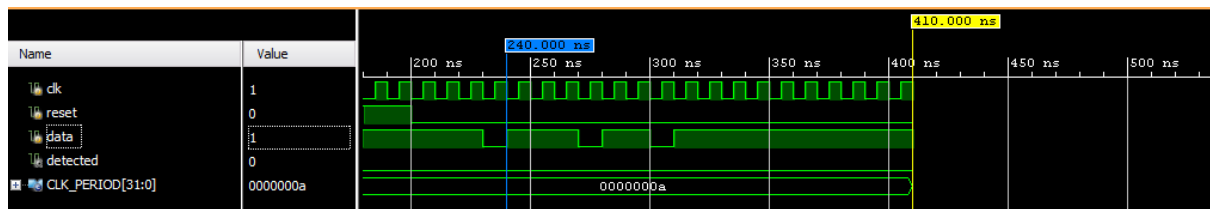
```
#100;
```

```
$finish;
```

```
end
```

```
endmodule
```

Waveform:



Conclusion:

In conclusion, we used a thorough behavioral testbench to validate both overlapping and non-overlapping scenarios after implementing a Mealy state machine for pattern identification in Verilog.

